Design Principles Behind Smalltalk

El proyecto Smalltalk busca brindar soporte computacional a la creatividad individual. Para ello, se enfoca en dos áreas clave:

1. **Lenguaje de descripción (programación):** Sirve de interfaz entre el pensamiento humano y el hardware.

2. **Lenguaje de interacción (interfaz de usuario):** Facilita la comunicación entre personas y computadoras.

El desarrollo sigue un ciclo inspirado en el método científico: construir una aplicación, rediseñar el lenguaje con base en la experiencia, y crear un nuevo sistema. Smalltalk-80 es el quinto resultado de este ciclo.

Principios fundamentales

- **Dominio personal:** El sistema debe ser comprensible por una sola persona.
- **Buen diseño:** Pocas partes inmutables, máxima generalidad y marco uniforme.
- **Propósito del lenguaje:** Facilitar la comunicación, tanto explícita como implícita
- **Alcance amplio:** Smalltalk abarca modelos internos, medios externos e interacción entre humanos y computadoras.

Objetos y mensajes

- **Objetos:** Deben reflejar cómo pensamos y distinguimos cosas. Cada objeto tiene una identidad y vida propia.
- **Manejo automático de memoria:** El sistema debe gestionar objetos automáticamente.
- **Mensajes:** Los objetos se comunican enviando mensajes, manteniendo encapsulamiento y claridad.
- **Metáfora uniforme:** Todo en el lenguaje debe seguir una misma lógica conceptual (como en Smalltalk, LISP o APL).

Organización del sistema

- **Modularidad:** Cada componente debe ser independiente para reducir la complejidad.

- **Clasificación:** El lenguaje debe permitir agrupar objetos similares mediante clases y facilitar la creación de nuevas.

- **Polimorfismo:** Los programas deben depender del comportamiento de los objetos, no de su tipo concreto.

- **Factorización:** Cada funcionalidad debe estar definida en un solo lugar, lo que facilita el mantenimiento y la reutilización.

- **Herencia:** Permite compartir y extender comportamientos entre clases.

- **Apalancamiento (leverage):** Un buen diseño maximiza la reutilización y extensión del sistema con mínimo esfuerzo.

Unit testing Guidelines

- **Pequeñas y rápidas:** Las pruebas deben ejecutarse rápidamente para permitir feedback inmediato y facilitar la integración continua.

- **Automatizadas y sin interacción:** Las pruebas deben ejecutarse automáticamente sin intervención humana.

- **Fáciles de ejecutar:** Ejecutar una prueba debe requerir un solo comando o clic.

- **Medir cobertura:** Usar herramientas de análisis de cobertura para saber qué partes del código se prueban realmente.

- **Reparar fallos de inmediato:** Todos deben priorizar arreglar pruebas rotas en cuanto fallan.

- **A nivel unidad:** Testear clases en aislamiento. No usar tests de flujo completo como unitarios.

- **Empezar simple:** Un test simple ya verifica el entorno y sirve como base.

- **Independientes:** Los tests no deben depender entre sí ni del orden de ejecución.

- **Cercanos al código:** Los tests deben estar en el mismo paquete/directorio que la clase que prueban.

- **Nombres claros:** El nombre del test debe reflejar la funcionalidad que verifica.
- **Testear API pública:** No probar métodos privados. Refactorizar si hace falta testear lógica oculta.

- **Caja negra:** Testear desde el punto de vista del consumidor externo.

- **Caja blanca:** También verificar las partes más complejas del código.

- **Probar lo trivial también:** Incluso los getters y setters pueden contener errores.

- **Cobertura de ejecución primero:** Asegurarse de que se ejecuta todo el código al menos una vez.

- **Casos límite:** Probar límites de valores, como `0`, negativos, extremos, cadenas vacías, etc.

- **Tests con datos aleatorios:** Mejoran cobertura al variar entradas en cada ejecución.

- **Un test por funcionalidad:** No mezclar muchas verificaciones en un solo test.

- **Asserts explícitos:** Usar *assertEquals(a, b)* en lugar de *assertTrue(a == b)* para mejor feedback.

- **Tests negativos:** Verificar que se lancen excepciones esperadas ante inputs inválidos.
- **Diseño testeable:** Minimizar API pública, evitar complejidad innecesaria, usar estado inmutable.
- **Sin dependencias externas:** No depender de archivos externos o bases de datos. Generar los datos en el test.
- **Costo del testing:** Apuntar a ~80% de cobertura. Más puede ser muy costoso, especialmente en errores excepcionales.
- **Priorizar tests:** Testear primero la lógica más baja o más crítica del sistema.
- **Resiliencia ante fallos:** Prever fallos para que un error no detenga toda la suite.
- **Tests para bugs:** Cada bug debe tener un test que lo reproduzca antes de arreglarlo.
- **Simplicidad:** Los tests no deben ser complejos ni duplicar la lógica del código que prueban.
- **Reconocer límites:** Las pruebas no garantizan ausencia de errores. Sirven como documentación, verificación y ayuda para refactorizar con confianza.

8 Principles of Better Unit Testing

Guía 1: Saber qué estás probando

- Un test debe tener un **objetivo claro**.
- Evita los tests largos y confusos que verifican muchas cosas a la vez.
- **Un solo escenario por test** mejora la comprensión y facilita el mantenimiento.
- Si te cuesta nombrar el test, probablemente no está bien enfocado.

Guía 2: Los tests deben ser autosuficientes

- Los tests **no deben depender del entorno** (base de datos, registros, orden de ejecución).
- Ejecutar un test 1,000 veces debería dar **el mismo resultado siempre**.
- Evitá estados globales o compartidos; si se usan, deben reiniciarse entre tests.

Guía 3: Los tests deben ser deterministas

- Un test que **a veces pasa y a veces falla** es inútil.
- No uses **datos aleatorios** ni condiciones dependientes del rendimiento o la máquina.
- Los fallos deben indicar de forma clara un error en el código, no una condición externa.

Guía 4: Convenciones de nombres

- El **nombre del test debe indicar** qué se prueba y cuál es el resultado esperado.

- Un buen nombre ayuda a entender rápidamente por qué falló el test.
- Es preferible tener varios tests con código similar pero bien nombrados que uno genérico difícil de entender.

Guía 5: Eliminación de duplicación vs legibilidad

- Está bien eliminar duplicación (factories, custom assertions,etc) **siempre y cuando no se sacrifique la claridad** del test.

Guía 6: Probar resultados, no la implementación

- Los tests deben fallar **solo si el comportamiento observable cambia**.
- Evitá probar métodos privados o detalles internos del código.
- Cambios triviales en la implementación **no deben romper tests** si el resultado final no cambió.

Guía 7: Evitar la sobre especificación

- No escribas tests que **micro manejen** la ejecución (ej: “el método debe llamarse exactamente 3 veces”).
- Eso hace los tests frágiles y difíciles de mantener.
- Permití cierta **flexibilidad** si el resultado esperado no cambia.

Guía 8: Usar un framework de aislamiento (mocking)

- Las dependencias internas o externas dificultan el testing.
- Usá frameworks de mocking para **evitar configuraciones complejas**.
- Los mocks automáticos reducen el código repetido y **aumentan la productividad**.

The Art of Enbugging

La mayoría de los errores en nuestros programas no aparecen de la nada. **Los bugs no surgen espontáneamente**, los introducimos nosotros, muchas veces sin darnos cuenta. Diseñar código que reduzca las probabilidades de error desde el principio es clave para mantener sistemas sostenibles.

Una estrategia poderosa es **escribir código “tímido” (shy code)**, que se mantiene aislado, encapsulado y sin meterse en los detalles internos de otros objetos.

1. Entender que los errores no son ajenos

Hablar de "bugs" suena simpático, pero **despersonaliza la responsabilidad**. La realidad es que:

- Nosotros ponemos los errores.
- Muchas veces no lo hacemos directamente, sino que **preparamos condiciones propensas a que aparezcan errores después**.

2. Separación de responsabilidades

Una buena práctica para evitar futuros errores es **diseñar con responsabilidades claras y bien aisladas**. Cada clase o módulo debe tener:

- Una tarea bien definida.
- Interfaces claras.
- Bajo acoplamiento con otros módulos.

Esto reduce el impacto de cambios y hace que el código sea más predecible.

3. Código tímido (Shy Code)

El **código tímido**:

- No se mete con otros innecesariamente.
- No expone su estado interno sin necesidad.
- Evita compartir más información de la necesaria.

Esto contrasta con el “vecino chismoso”: código que mira, toca o decide sobre estados internos de otros objetos.

4. “Tell, Don’t Ask”

En diseño orientado a objetos, en lugar de consultar un estado y luego actuar, se recomienda **darle la orden al objeto directamente**.

Consejo: Clasificá tus métodos como:

- **Comandos:** cambian el estado (pueden devolver algo).
- **Consultas:** devuelven información sin efectos secundarios visibles.

Esta separación mejora pruebas, mantenimiento y legibilidad.

5. La Ley de Deméter

Mientras menos objetos contactes directamente, menor el riesgo de errores.

Esta ley sugiere que un objeto debe interactuar solo con:

- Sí mismo.
- Parámetros del método.
- Objetos que crea internamente.
- Componentes directamente referenciados.

6. Delegar sin miedo: costo y beneficio

A veces implica escribir muchos métodos simples que solo delegan, pero:

- El **acoplamiento bajo** mejora la estabilidad del sistema.

- El pequeño costo en rendimiento se compensa con una gran ganancia en mantenibilidad.

Tell Dont ask

El principio **Tell-Don't-Ask** ayuda a recordar que la orientación a objetos trata de combinar datos con las funciones que operan sobre esos datos. En lugar de **preguntar a un objeto por sus datos y actuar sobre ellos desde afuera**, deberíamos **decirle al objeto qué hacer**. Esto fomenta mover el comportamiento hacia el propio objeto, junto con sus datos.

Muchas personas encuentran útil este principio porque uno de los fundamentos del diseño orientado a objetos es que los objetos combinen datos y comportamiento. Esto es beneficioso porque el comportamiento y los datos están estrechamente relacionados: cambiar uno suele implicar cambiar el otro, y entender uno ayuda a comprender el otro. Por eso, elementos estrechamente ligados deben estar en el mismo componente.

Tell-Don't-Ask es una manera de ayudar a los programadores a aumentar esta co-localización de datos y comportamiento.

Sin embargo, personalmente, no siempre sigo el principio al pie de la letra. Aunque busco co-localizar datos y comportamiento (lo que muchas veces conduce a resultados similares), me preocupa que Tell-Don't-Ask lleve a algunos a eliminar todos los métodos de consulta (getters), lo cual no siempre es deseable.

Hay situaciones donde los objetos colaboran eficientemente proporcionando información, especialmente cuando transforman o simplifican datos para sus clientes. En estos casos, eliminar consultas puede complicar el código innecesariamente.

En resumen, Tell-Don't-Ask es un buen punto de partida para fomentar la co-localización del comportamiento con los datos, pero no es un principio que deba seguirse ciegamente en todas las situaciones.

Getter Erradicator

El rechazo a los métodos getter, suele basarse en la idea de que los **getters violan la encapsulación**. Por ejemplo, si una clase tiene campos y se añaden métodos para obtener directamente esos valores, esto se asemeja mucho a hacer los campos públicos, lo que va contra el principio de ocultar detalles internos.

Sin embargo, la verdadera esencia de la encapsulación no es simplemente ocultar datos, sino **ocultar decisiones de diseño que puedan cambiar en el futuro**. No se trata solo de proteger la representación interna de datos, sino también de **ocultar protocolos o detalles sobre cómo un objeto se comunica o realiza ciertas tareas**. Por eso, al pensar en encapsulación, es más útil preguntarse qué variabilidad se está ocultando y por qué, en lugar de simplemente si se están exponiendo datos.

Aunque la defensa de la encapsulación es la razón común para evitar getters, la motivación real suele ser pragmática. Muchas veces el código orientado a objetos termina escribiéndose con estilo procedural, sacando datos de un objeto para operar sobre ellos externamente, cuando el comportamiento debería residir dentro del propio objeto. Esto va contra el principio “Tell Don’t Ask” y usar getters facilita esa práctica procedural. Por eso, eliminar getters a menudo se propone como una forma de fomentar una mejor distribución del comportamiento.

Sin embargo, evitar todos los getters de forma rígida no es una solución ideal. En la práctica, los objetos necesitan colaborar intercambiando datos, y en esos casos, los getters son necesarios y útiles.

Como regla práctica, Kent Beck aconseja prestar atención cuando un código invoca múltiples métodos sobre el mismo objeto, ya sea getters o comandos. Pregúntate si esas múltiples llamadas pueden reemplazarse por una sola solicitud o comando que haga el objeto más autónomo. Esto ayuda a reducir la dependencia excesiva y mejora la cohesión.

Otro indicio de problemas es la presencia de “Data Classes”, clases que sólo contienen datos y getters sin comportamiento. Esto suele indicar que el diseño no está aprovechando bien la orientación a objetos y que el comportamiento debería moverse hacia esas clases.

En definitiva, asignar comportamiento y datos es el núcleo del diseño orientado a objetos, y no hay reglas absolutas, sino que hay que valorar compensaciones. La primera opción suele ser colocar el comportamiento junto con los datos (el enfoque “Information Expert”), pero no es la única. Algunos patrones de diseño separan datos y comportamiento por razones justificadas. Un buen criterio general es que lo que cambia junto debería estar junto: los datos y el comportamiento asociado suelen cambiar juntos, aunque también existen otras agrupaciones importantes.

Nota al pie: un uso claro de getters sería cuando en un juego de cartas (como en el tp2), pedís del modelo las cartas de una mano, para poder mostrar al usuario las cartas. Ese getter, es muy difícil de no usarse. Más adelante, cuando vean patrones de diseño, van a ver un patrón que se llama *Observer*, muy usado para lo que es juegos y ayuda a mitigar el abuso de getters, pero no los erradica por completo.

¿Para Qué Sirve un Modelo UML?

El uso del UML (Unified Modeling Language) es considerado por muchos como uno de los avances más importantes en los últimos años, especialmente por resolver conflictos sobre notación. Sin embargo, queda la cuestión fundamental: ¿para qué sirve realmente el UML? ¿Por qué usarlo, o por qué no?

Crear un diagrama UML tiene un coste, y un modelo UML no es algo que importe directamente al cliente. Los clientes quieren software que funcione, no diagramas, por bonitos o estándar que sean. Por lo tanto, el valor del modelado está ligado al impacto que tiene en la calidad o coste del software. Si el modelo mejora la calidad o reduce el coste, tiene valor; si no, no.

El valor principal de un modelo es proporcionar una mejor comprensión, ya sea del software o del dominio que soporta, a través de representaciones gráficas o resaltando información clave. Un reto es que las personas piensan y entienden de formas distintas: algunos prefieren texto, otros gráficos, y esta preferencia puede variar según el tema. Por ello, imponer estándares rígidos puede ser contraproducente; lo ideal es encontrar la combinación que funcione para el equipo.

Existen dos enfoques principales para modelar:

1. **Modelo detallado o completamente desarrollado:** Consiste en convertir todo el texto o detalles en el modelo, reflejando toda la información posible. Esto es laborioso y sólo rentable si se cuenta con herramientas que automatizan la tarea (por ejemplo, herramientas CASE con sincronización entre código y modelo). Sin embargo, en estos modelos muy detallados, la claridad puede perderse, especialmente en aspectos como el flujo de control.

2. **Modelo esquelético o resumido:** Se centra en resaltar solo las partes importantes del sistema, evitando el exceso de detalles que dificultan la comprensión. Este tipo de modelo muestra “los huesos” del sistema y es más fácil de entender y mantener, ya que los detalles importantes cambian menos y el modelo es más útil para guiar la exploración en el código, que siempre está actualizado.

La clave está en que un modelo no puede ser a la vez completamente detallado y esquelético; debe elegirse uno de los dos enfoques. Esta elección debe ser consciente y basada en qué se considera más valioso para visualizar el sistema. Las herramientas CASE tienden a favorecer modelos completamente desarrollados, ya que se apoyan en la estructura de datos de las clases y no en sus interfaces, lo que puede exponer detalles que deberían estar ocultos.

Por tanto, para comunicar eficazmente, un modelo esquelético es preferible, aunque requiere esfuerzo manual. En última instancia, la comprensión humana es insustituible.

Replace Conditional with Polymorphism

Problema
Tienes un condicional que realiza diferentes acciones según el tipo de objeto o sus propiedades.

Solución
Crea subclases que correspondan a cada rama del condicional. En ellas, define un método común y mueve el código correspondiente de cada rama del condicional a ese método. Luego, reemplaza el condicional por una llamada al método adecuado. De este modo, la implementación correcta se obtendrá a través del polimorfismo, dependiendo de la clase del objeto.

Por qué refactorizar
Este patrón es útil cuando tu código tiene operadores que realizan tareas diferentes según:

- La clase del objeto o la interfaz que implementa.
- El valor de algún campo del objeto.
- El resultado de invocar un método del objeto.

Si aparece una nueva propiedad o tipo de objeto, tendrías que buscar y modificar todos los condicionales similares, lo que puede ser costoso y propenso a errores. Usar subclases para el comportamiento elimina esta dispersión y facilita la extensibilidad.

Beneficios

- Sigue el principio *Tell-Don't-Ask*: en lugar de preguntar por el estado del objeto y luego decidir qué hacer, simplemente le dices al objeto qué hacer y él decide cómo.
- Elimina código duplicado al evitar múltiples condicionales similares.
- Facilita añadir nuevas variantes de comportamiento creando nuevas subclases, sin modificar el código existente (principio Abierto/Cerrado).

Cómo refactorizar

1. Prepara una jerarquía de clases que representen las distintas variantes de comportamiento. Si no tienes una, crea una. Algunas técnicas para esto son:
    - Reemplazar código de tipo con subclases, creando una subclase por cada valor relevante.
    - Reemplazar código de tipo con estado o estrategia, donde el objeto delega el comportamiento a una clase especializada.
2. Si el condicional está dentro de un método que realiza otras acciones, primero extrae el condicional en un método separado.
3. Para cada subclase, redefine el método que contiene el condicional y copia el código que corresponde a esa rama del condicional.
4. Elimina la rama correspondiente del condicional original.
5. Repite hasta que el condicional quede vacío, entonces elimínalo y convierte el método en abstracto para que cada subclase implemente su versión.

¿Cuál es el propósito del Desarrollo Guiado por Pruebas (TDD)? Capítulo 1

1. El desarrollo de software es un proceso de aprendizaje continuo

- Casi todos los proyectos de software intentan algo nuevo, combinando personas, tecnología y dominio desconocido.
- Los desarrolladores no siempre conocen completamente las tecnologías o el dominio, y deben aprender mientras trabajan.
- Los clientes y usuarios también descubren detalles sobre sus procesos y necesidades durante el proyecto.

- Por eso, el proceso debe ser flexible para manejar incertidumbres y cambios inesperados.

2. El feedback es la herramienta fundamental

- La mejor forma de avanzar es aplicar ciclos cortos y repetidos de trabajo con retroalimentación constante.
- Cada ciclo incluye análisis, diseño, implementación y despliegue de funcionalidades.

- Desplegar el software frecuentemente permite confrontar las suposiciones con la realidad, detectar errores y ajustar el plan.

- Estos ciclos forman una red de “loops” desde segundos (ej. tests unitarios) hasta meses (ej. entregas completas).

- Los ciclos internos se centran en detalles técnicos y calidad del código; los externos en la satisfacción del cliente y la efectividad del equipo.

- Cuanto antes se reciba feedback, mejor.

3. Desarrollo incremental e iterativo

- Se desarrolla la aplicación por funcionalidades completas, no por capas aisladas.
- El sistema siempre está integrado y listo para desplegar.
- Las funcionalidades se refinan iterativamente con base en el feedback.

4. Prácticas para soportar el cambio

- Dos bases técnicas imprescindibles:

    1. **Pruebas constantes:** para detectar errores de regresión, automatizando la mayor cantidad posible para reducir costos.

    2. **Código simple:** fácil de entender y modificar, logrado mediante refactorización continua.

- El código debe ser legible, ya que se lee mucho más de lo que se escribe.
- Las pruebas automatizadas protegen contra errores durante el cambio.
- Sin embargo, la escritura de pruebas no es muy popular entre desarrolladores, pues suele verse como trabajo “menos real” o aburrido.

5. TDD cambia la forma de trabajar con las pruebas

- En lugar de probar después de codificar, se escribe primero la prueba que falla.

- Esto convierte las pruebas en una actividad de diseño que clarifica qué debe hacer el código.

- Al escribir pruebas antes, se recibe rápido feedback sobre la calidad del diseño.

- Esto también ayuda a que el código sea más modular y fácil de testear.

- Se crea una red de pruebas que dan confianza para cambiar el código sin miedo a romper funcionalidades.

- **Regla de oro del TDD:** Nunca escribir funcionalidad nueva sin antes tener una prueba que falle.

6. El ciclo fundamental de TDD

- Escribir una prueba que falla.
- Escribir código para que la prueba pase.
- Refactorizar para simplificar y mejorar el código.
- Repetir.

7. Refactorización

- Cambiar la estructura interna del código sin alterar su comportamiento.
- Mejorar la mantenibilidad y claridad del código.
- Se realizan pequeños pasos seguros, con tests para asegurar que no se rompe nada.

8. El panorama completo: pruebas unitarias, de integración y de aceptación

- No basta con pruebas unitarias; hay que asegurarse que el sistema completo funciona y es útil para el usuario.
- Las pruebas de aceptación simulan el uso real y guían la implementación.
- Las pruebas de integración verifican que el código funciona con librerías externas o módulos no modificables.
- La calidad externa (funcionalidad, confiabilidad) y la interna (facilidad para cambiar y entender el código) son igual de importantes.

9. Pruebas end-to-end (de punta a punta)

- Deben ejercitar el sistema completo desde la interfaz de usuario o puntos externos, no sólo el código interno.
- Validan la interacción real con el entorno y los procesos de despliegue.
- Son críticas porque a menudo la mayor complejidad está en la interacción con el entorno.